

PROJECT-SPECIFIC TECHNICAL REFERENCE

Spring Boot Banking Project Interview Guide

Theory, request flows, tradeoffs, likely questions, and strong answers

How to use this guide

Learn the concepts, then explain each one using the actual classes in your project. Interviewers value accurate tradeoffs and honest limitations more than memorized definitions.

Current project scope: JWT authentication • account management • transactional money movement • fraud detection • audit logging • React frontend

1. Thirty-Second Project Introduction

Use this when the interviewer asks: “Tell me about your project.”

Strong answer

I built a layered banking transaction simulator using Spring Boot and React. The backend exposes REST APIs secured with Spring Security and JWT. It models users, accounts, transactions, fraud flags, and audit logs using Spring Data JPA. Money movement is atomic through @Transactional, and account rows are pessimistically locked to prevent concurrent withdrawals from overspending. The application also applies configurable fraud rules, role-based admin authorization, DTO validation, and centralized exception handling.

What the Project Demonstrates

Area	Current implementation	Spring concept demonstrated
Authentication	Register, login, BCrypt password hashing, JWT	Spring Security, AuthenticationManager, filters
Authorization	Authenticated routes and ADMIN-only endpoints	SecurityFilterChain, roles, SecurityContext
Persistence	Users, accounts, transactions, flags, logs	JPA entities, relationships, repositories
Money movement	Deposit, withdrawal, account-to-account transfer	Transactions, locking, BigDecimal
Fraud controls	Amount, count, and daily-total rules	Service rules, JPQL aggregation, configuration
API quality	DTOs, validation, consistent error JSON	Bean Validation, @RestControllerAdvice
Frontend	React banking dashboard served by Spring Boot	REST integration and static resources

2. Architecture and Separation of Concerns

The application uses a layered architecture. Each layer has a focused responsibility, which reduces coupling and makes changes and tests easier.

```
React UI → REST Controller → Service → Repository → Database
                        ↘ Security, validation, exceptions, audit logging
```

Responsibilities by Layer

Layer	Responsibility	Examples
Controller	Maps HTTP requests, validates request bodies, reads Principal, returns responses.	AuthController, AccountController, TransactionController

Service	Enforces business rules and coordinates repositories.	AuthService, TransactionService, FraudDetectionService
Repository	Provides database access through Spring Data JPA.	AccountRepository, TransactionRepository
Entity	Maps domain objects and relationships to database tables.	User, Account, Transaction, FraudFlag
DTO	Defines safe and stable API request/response contracts.	TransferRequest, AccountResponse

Interview principle

Controllers should be thin. Business rules belong in services so they are reusable, transactional, and testable without HTTP.

Dependency Injection and the Spring Container

Spring creates and manages application objects called beans. Classes annotated with `@Service`, `@Component`, `@Configuration`, or `@RestController` are discovered during component scanning. Constructor injection supplies their dependencies.

- `@RequiredArgsConstructor` generates a constructor for final fields.
- Constructor injection makes required dependencies explicit and supports unit testing.
- Spring manages bean lifecycle and can wrap beans with proxies for features such as `@Transactional`.

Likely Interview Questions

Question	Strong short answer
Why not call repositories directly from controllers?	It mixes HTTP and business logic, complicates testing, and makes transaction boundaries unclear.
What is inversion of control?	The framework creates and wires objects instead of application code manually constructing every dependency.
Why prefer constructor injection?	Dependencies are explicit, immutable, and easy to replace with mocks in tests.

3. REST APIs, DTOs, and Validation

REST Controller Theory

@RestController combines @Controller and @ResponseBody. Returned Java objects are serialized to JSON by Jackson. @RequestMapping defines a common route prefix, while @GetMapping and @PostMapping map individual operations.

- POST /api/auth/register creates a user.
- GET /api/accounts returns accounts belonging to the authenticated user.
- POST /api/transactions/transfer performs a transfer.

The controllers receive Principal. Its name comes from the authenticated user stored in Spring Security's SecurityContext, so clients do not choose which user they are acting as.

Why DTOs Matter

DTOs separate the external API contract from persistence entities. Request DTOs control accepted fields; response DTOs control exposed fields.

- Prevents password hashes and internal relationships from being serialized.
- Avoids clients changing protected entity fields such as role, balance, or owner.
- Allows database entities to evolve without automatically breaking the API.
- Avoids recursive JSON serialization across bidirectional relationships.

Bean Validation Flow

1. The controller receives JSON and Jackson creates a request DTO.
2. @Valid triggers Jakarta Bean Validation annotations on the DTO.
3. Invalid input throws MethodArgumentNotValidException before service execution.
4. GlobalExceptionHandler converts validation errors into consistent JSON.

Useful distinction

Validation checks input shape and simple constraints. Business rules such as sufficient balance, ownership, or active status belong in the service layer.

4. Authentication with Spring Security and JWT

Registration Flow

5. AuthService checks whether the email already exists.
6. PasswordEncoder hashes the password using BCrypt.
7. The user is saved through UserRepository.
8. An audit log is recorded.
9. JwtUtil generates a signed token returned to the frontend.

Login Flow

```
LoginRequest → AuthenticationManager → AuthenticationProvider
               → UserDetailsService → UserRepository
               → PasswordEncoder comparison → JWT generation
```

AuthenticationManager does not itself verify passwords. It delegates to an AuthenticationProvider. The configured DaoAuthenticationProvider loads UserDetails and uses BCryptPasswordEncoder to compare the submitted password with the stored hash.

Why BCrypt?

- It is a one-way password hashing algorithm, not encryption.
- It automatically includes a random salt, protecting identical passwords from identical hashes.
- Its work factor makes brute-force attempts expensive.
- Passwords are verified by hashing the candidate and comparing, never by decrypting.

JWT Request Flow

10. The frontend sends Authorization: Bearer <token>.
11. JwtAuthFilter runs once per request.
12. It verifies the token signature and expiration.
13. It loads the user and creates an authenticated UsernamePasswordAuthenticationToken.
14. Authentication is stored in SecurityContextHolder.
15. Spring Security authorizes the endpoint and Principal becomes available.

JWT Benefits and Tradeoffs

Benefits	Tradeoffs
Stateless and easy to scale across multiple server instances.	Harder to revoke immediately before expiration.
Useful for REST APIs and separate frontend clients.	Tokens must be stored and transmitted securely.
No server-side session lookup on every request.	Token claims can become stale if roles change.

Honest improvement

The current frontend stores JWT in localStorage, which is convenient but exposed to XSS. A production design may use short-lived access tokens plus rotating refresh tokens in secure HttpOnly cookies.

5. Authorization and Ownership

Authentication answers “Who are you?” Authorization answers “What are you allowed to do?” The project applies authorization at both route and service levels.

Route-Level Authorization

- Static frontend files and `/api/auth/**` are public.
- `/api/admin/**` requires the ADMIN role.
- All remaining routes require authentication.

`CustomUserDetailsService` converts the project’s Role enum into Spring authorities. `roles("ADMIN")` produces `ROLE_ADMIN`, which matches `hasRole("ADMIN")`.

Object-Level Authorization

Route protection is not enough. A logged-in user must not withdraw from another user’s account. `AccountService` and `TransactionService` verify that the source account owner email equals the authenticated email.

Security design choice

Unauthorized accounts return an account-not-found response. This reduces account enumeration because the API does not reveal whether another user’s account exists.

Current Cross-Holder Transfer Behavior

The backend already allows transferring to another account holder. It verifies ownership of the source account, but the destination only needs to exist and be active. The current weakness is that the UI asks for an internal database ID.

Check	Source account	Destination account
Must exist	Yes	Yes
Must be active	Yes	Yes
Must belong to logged-in user	Yes	No
Loaded with a write lock	Yes	Yes

The planned beneficiary feature should use public account numbers, verify recipient details, enforce transfer limits, and save approved beneficiaries.

6. JPA Entities and Database Modeling

JPA maps Java entities to relational tables. Hibernate is the JPA implementation used by Spring Boot.

Current Domain Model

Entity	Important relationships and purpose
User	Owns many accounts; stores email, BCrypt password hash, role, and creation time.
Account	Belongs to one user; stores unique account number, type, status, and balance.
Transaction	References optional source and destination accounts; stores type, status, amount, and description.
FraudFlag	References a flagged transaction and records reason and resolution state.
AuditLog	Records actor, action, resource, details, and timestamp.

Important JPA Choices

- `@Entity` marks persistent domain classes; `@Table` defines table names.
- `@Id` and `@GeneratedValue(strategy = IDENTITY)` use database-generated primary keys.
- `@ManyToOne(fetch = LAZY)` avoids loading related objects until needed.
- `@OneToMany(mappedBy = "owner")` makes `Account.owner` the owning side of the relationship.
- `EnumType.STRING` stores `ACTIVE` or `SAVINGS` instead of fragile numeric enum positions.
- Column precision 19 and scale 2 model currency values safely.

Lazy Loading and the N+1 Problem

Lazy loading reduces unnecessary data retrieval, but repeatedly accessing lazy relationships across a list may trigger many additional queries. This is called the N+1 query problem. It can be addressed with fetch joins, entity graphs, projections, or carefully designed repository queries.

Spring Data JPA Repositories

Repositories extend `JpaRepository`, which provides CRUD and pagination APIs. Spring Data can derive queries from method names.

```
findByOwnerEmail(email)
findBySourceAccountIdOrDestinationAccountIdOrderByCreatedAtDesc(...)
existsByAccountNumber(accountNumber)
```

The fraud service also uses custom JPQL to calculate total outgoing money during a time window.

7. Transactions and Money Movement

Why @Transactional Is Essential

@Transactional defines a database transaction boundary. If an unchecked exception escapes, Spring rolls back the entire operation.

Transfer atomicity

A transfer must subtract from the source, add to the destination, and save the transaction record as one atomic unit. Partial success would corrupt financial data.

ACID Applied to This Project

Property	Meaning in the banking project
Atomicity	Every part of a transfer succeeds or all changes roll back.
Consistency	Rules such as sufficient balance and active account status preserve valid state.
Isolation	Locks prevent concurrent operations from using stale balances.
Durability	Once committed, transaction and balance updates remain persisted.

Deposit Flow

16. Lock and load the user-owned active destination account.
17. Add the amount using BigDecimal.
18. Create a COMPLETED DEPOSIT transaction.
19. Record an audit entry and commit.

Withdrawal Flow

20. Lock and load the user-owned active source account.
21. Check that balance is greater than or equal to the requested amount.
22. Subtract using BigDecimal and save a COMPLETED WITHDRAWAL transaction.
23. Record an audit entry and commit.

Transfer Flow

24. Reject identical source and destination account IDs.
25. Lock and validate the owned source account.
26. Lock and validate the destination account.
27. Check sufficient funds.
28. Evaluate fraud rules.
29. If safe, update both balances and save a COMPLETED transaction.

30. If suspicious, save a FLAGGED transaction and FraudFlag without moving money.

Why BigDecimal Instead of double?

Binary floating-point values cannot exactly represent many decimal amounts. That can produce rounding errors in currency calculations. BigDecimal represents decimal values precisely and supports explicit rounding policies.

```
if (account.getBalance().compareTo(amount) < 0) {  
    throw new InsufficientFundsException("Insufficient funds");  
}
```

Important Spring Proxy Detail

@Transactional normally works through a Spring proxy. Calling a transactional method from another method inside the same class can bypass the proxy, so the transaction annotation may not take effect. Transaction boundaries should usually be on externally invoked public service methods.

8. Concurrency and Pessimistic Locking

Without concurrency control, two simultaneous withdrawals could both read a ₹1,000 balance and each withdraw ₹800, resulting in an invalid negative balance.

```
@Lock(LockModeType.PESSIMISTIC_WRITE)  
Optional<Account> findWithLockById(Long id);
```

PESSIMISTIC_WRITE asks the database to lock the selected row until the current transaction commits or rolls back. Other writers must wait.

Tradeoffs

Advantages	Risks and mitigations
Simple and strong protection for balance updates.	Reduced throughput when many requests target the same account.
Prevents stale reads during financial writes.	Deadlocks are possible when rows are locked in inconsistent order.
Works naturally inside @Transactional methods.	Lock timeouts and retry policies may be required.

Improvement to discuss

For transfers, lock accounts in a deterministic order, such as smaller account ID first, to reduce deadlock risk when two users transfer in opposite directions.

9. Fraud Detection

FraudDetectionService evaluates transfers before balances are changed. Thresholds come from application properties, so operators can tune rules without recompiling the application.

Current Rules

- Single transfer amount exceeds the configured large-transfer threshold.
- Outgoing transfer count during the last 24 hours exceeds a configured maximum.
- Outgoing total during the last 24 hours plus the requested transfer exceeds a configured threshold.

Why noRollbackFor Matters

```
@Transactional(noRollbackFor = FraudDetectedException.class)
```

Normally, FraudDetectedException is an unchecked exception and would roll back the transaction. noRollbackFor intentionally commits the FLAGGED transaction, FraudFlag, and audit log while still returning an error response and leaving balances unchanged.

Strong answer

Fraud rejection is not merely an error; it is an auditable business outcome. Therefore the fraud evidence must persist even though the transfer does not complete.

Fraud System Improvements

- Score transactions instead of returning only yes/no rules.
- Detect transfers to newly added beneficiaries.
- Check unusual time, location, device, or velocity.
- Create review workflows to resolve flags.
- Publish fraud-review events asynchronously.

10. Audit Logging

Audit logs provide traceability for actions such as registration, login, account creation, deposits, withdrawals, transfers, and fraud flags.

Field	Question answered
actorEmail	Who performed the action?
action	What happened?
resource	What type of resource was affected?
details	What contextual information is useful?

createdAt	When did it occur?
-----------	--------------------

Audit logs support investigations, debugging, compliance, and accountability.

Current limitation

AuditLogService participates in the caller's transaction. If the main operation rolls back, its audit entry may also roll back. A production design may use REQUIRES_NEW or an event/outbox approach, depending on whether failed attempts must always be retained.

11. Global Exception Handling

@RestControllerAdvice applies exception-handling logic across controllers. This prevents duplicated try/catch blocks and gives clients a consistent error structure.

```
{
  "timestamp": "...",
  "status": 400,
  "error": "Bad Request",
  "message": "Insufficient funds"
}
```

Current Mappings

Exception	HTTP behavior
MethodArgumentNotValidException	400 with field validation details
BadCredentialsException	401 with safe invalid-credentials message
AccountNotFoundException	400 in current implementation
InsufficientFundsException	400
FraudDetectedException	400
Unexpected Exception	500 without exposing internal details

Improvement to discuss

AccountNotFoundException would more naturally map to 404. FraudDetectedException may map to 409 Conflict or a domain-specific accepted-for-review response. Precise status design improves API semantics.

12. Configuration, Environments, and Infrastructure

application.properties uses environment-variable fallbacks, allowing the same application artifact to run with different infrastructure and security settings.

- H2 provides a fast in-memory local-development database.
- PostgreSQL provides realistic persistent relational storage.
- Docker Compose starts PostgreSQL and Redis locally.
- JWT expiration and fraud thresholds are externally configurable.
- RedisTemplate is configured with string serializers.

Be accurate in interviews

Redis is configured, but no current business functionality uses it. Present it as infrastructure prepared for future caching, token revocation, rate limiting, or distributed locks, not as a completed feature.

Useful Redis Features to Add

- Rate-limit login and transfer attempts.
- Store revoked JWT identifiers until expiration.
- Cache frequently requested non-financial reference data.
- Implement short-lived OTP or beneficiary verification codes.

13. Frontend and Backend Integration

The React frontend calls the Spring Boot REST APIs. During development, Vite proxies /api requests to Spring Boot. For production, the React build is written into Spring Boot's static resources directory and served from the same application.

- Same-origin production hosting simplifies deployment and avoids CORS configuration.
- The frontend attaches the JWT Bearer token to authenticated requests.
- The UI does not replace backend authorization; the backend must enforce every rule.

Interview principle

Client-side checks improve user experience, but they are never trusted security controls. Attackers can call APIs directly.

14. Current Limitations and Honest Improvement Answers

Current limitation	Production-grade improvement
Destination entered using internal database ID	Use account number lookup plus approved beneficiaries.
First registered user automatically becomes ADMIN	Provision admins through secure deployment or controlled admin workflow.
JWT stored in localStorage	Use short-lived tokens and secure HttpOnly refresh cookies.
No idempotency key for transfers	Reject duplicate retries using a unique idempotency record.
Potential deadlocks from account lock order	Lock source/destination in deterministic account-ID order.
No pagination for transactions or audit logs	Use Pageable, sorting, and filtered repository specifications.
Redis configured but unused	Add rate limiting, token revocation, or OTP verification.
Minimal automated tests	Add service, repository, security, and integration tests.

15. Recommended Next Feature: Beneficiaries and External Transfers

This is the strongest next feature because it makes cross-holder transfers realistic while showcasing additional Spring Boot capabilities.

Proposed Domain Model

Component	Responsibility
-----------	----------------

Beneficiary entity	Stores owner, recipient account, nickname, status, creation time, and activation time.
BeneficiaryRepository	Finds active beneficiaries owned by the authenticated user.
BeneficiaryService	Adds, verifies, activates, lists, and removes beneficiaries.
BeneficiaryController	Exposes authenticated REST endpoints.
Transfer update	Accepts beneficiary ID instead of raw internal account ID.

Security and Business Rules

- Users may only view and use their own beneficiaries.
- Recipient lookup uses a public account number, never an internal ID.
- The API returns masked recipient details before confirmation.
- New beneficiaries may have an activation delay.
- Per-transfer and daily limits are checked.
- Adding and using beneficiaries creates audit records.
- Transfers remain atomic and account rows remain locked.

Potential Request Flow

```

POST /api/beneficiaries
  → validate public account number
  → reject own account if desired
  → create PENDING beneficiary
  → verify or wait for activation

POST /api/transfers/beneficiary
  → verify beneficiary ownership and ACTIVE status
  → lock accounts → fraud checks → transfer → audit

```

16. Testing Strategy

A strong project is not only feature-rich; it proves behavior with tests.

Test type	What to verify
Unit tests	Service rules using mocked repositories: insufficient funds, ownership, fraud threshold.
Repository tests	Derived queries, JPQL aggregation, locking queries, relationship persistence.
Controller tests	Validation, JSON structure, HTTP status, authenticated Principal behavior.
Security tests	Public routes, unauthorized requests, ADMIN-only endpoints, invalid JWT.
Integration tests	Real transfer atomicity and rollback using a test database.

Concurrency tests	Simultaneous withdrawals cannot overspend an account.
-------------------	---

17. Rapid-Fire Interview Questions

Question	Answer
What does <code>@SpringBootApplication</code> do?	It combines configuration, component scanning, and auto-configuration, and marks the main bootstrapping class.
What is Spring Boot auto-configuration?	Spring conditionally configures beans based on classpath dependencies, properties, and existing beans.
What is a bean?	An object created, configured, and managed by the Spring container.
What does <code>@Transactional</code> do?	Through a Spring proxy, it opens or joins a database transaction and commits or rolls back based on completion.
Why does unchecked exception usually trigger rollback?	Spring's default transaction policy rolls back for <code>RuntimeException</code> and <code>Error</code> .
What is pessimistic locking?	The database locks selected rows so concurrent writers wait until the transaction finishes.
Why use <code>BigDecimal</code> ?	It represents decimal currency values precisely, unlike binary floating-point types.
What is <code>UserDetailsService</code> ?	A Spring Security contract that loads user credentials and authorities by username.
Authentication vs authorization?	Authentication establishes identity; authorization decides permitted actions.
Why DTOs?	They create controlled API contracts and prevent exposing or binding persistence entities directly.
What is lazy loading?	Related data is loaded only when accessed, reducing unnecessary retrieval but potentially causing N+1 queries.
Why use <code>@RestControllerAdvice</code> ?	It centralizes exception-to-HTTP-response handling across REST controllers.
What is idempotency?	Repeating the same request produces no additional side effect; crucial for safely retrying financial operations.
How would you scale the application?	Use PostgreSQL, stateless API instances, externalized config, short-lived JWTs, Redis for shared controls, observability, and idempotent operations.

18. Final Interview Checklist

- Explain the request flow from controller to database without looking at code.
- Explain why `@Transactional` and pessimistic locking are both needed.

- Explain why the destination may belong to another user but the source must not.
- Explain JWT validation and how SecurityContext provides Principal.
- State honestly that Redis is configured but not yet used.
- Mention current limitations before the interviewer discovers them.
- Propose beneficiaries, idempotency, pagination, and testing as next improvements.

Best closing sentence

The project's strongest backend qualities are transactional consistency, service-level authorization, concurrency protection, and auditable fraud handling. My next step is to make external transfers production-like through beneficiaries, idempotency, and stronger automated testing.